

Fiche de Révision

Programmation Orientée Objet avec JavaScript ES6 & Node.js

Zineb CHERRADI

3ème année Génie Logiciel
ESISA

3 juin 2026

Table des matières

Introduction	2
1 Les Modules ES6	2
1.1 Export et Import	2
1.1.1 Export par défaut (default)	2
1.1.2 Export nommé	2
2 Programmation Orientée Objet (POO)	3
2.1 Classes et Constructeurs	3
2.2 Propriétés Privées (Encapsulation)	3
2.3 Getters et Setters	4
2.4 Membres Statiques	5
2.5 Héritage	5
3 Architecture en Couches	6
3.1 Structure du Projet	6
3.2 Couche Repository (Accès aux données)	6
3.2.1 Repository MySQL	6
3.2.2 Repository JSON	7
3.3 Couche Service (Logique métier)	7
3.4 Couche Configuration (BDD)	7
4 Programmation Asynchrone	8
4.1 Callbacks, Promises et Async/Await	8
4.2 Les Promises	8
4.3 Async/Await	8
5 Gestion des Erreurs et Fermeture	9
5.1 Gestion des erreurs avec .catch()	9
6 Fonction Constructeur (Avant ES6)	9
7 Récapitulatif des Concepts	10
8 Exemples d'Utilisation	10
8.1 Création d'objets	10
8.2 Appel aux services	11
9 Points Importants à Retenir	11
Conclusion	11

Introduction

Cette fiche de révision couvre tous les concepts abordés dans le cours de POO avec JavaScript ES6 et Node.js. Elle est structurée pour faciliter la compréhension et la révision rapide avant l'examen.

1 Les Modules ES6

1.1 Export et Import

En JavaScript ES6, les modules permettent d'organiser le code en plusieurs fichiers. Il existe deux types d'export :

1.1.1 Export par défaut (default)

Export par défaut

- Un seul export par fichier
- Importable sans accolades {}
- On peut le renommer à l'importation

```
1 // models/Author.js
2 export default class Author {
3   constructor(id, name, yearBorn) {
4     this.id = id;
5     this.name = name;
6     this.yearBorn = yearBorn;
7   }
8 }
```

Listing 1 – Export par défaut - Classe Author

```
1 // examples.js
2 import Author from './models/Author.js';
3 let a1 = new Author(101, "Brendan Eich", 1961);
```

Listing 2 – Import par défaut

1.1.2 Export nommé

Export nommé

- Plusieurs exports possibles par fichier
- Importable avec accolades {}
- Doit garder le même nom à l'importation

```
1 // models/Point.js
2 export function title() {
3   console.log('POO avec ES-6');
4 }
```

Listing 3 – Export nommé - Fonction

```
1 // exemples.js
2 import Point, {title} from './models/Point.js';
3 title(); // Appel de la fonction export e
```

Listing 4 – Import nommé

2 Programmation Orientée Objet (POO)

2.1 Classes et Constructeurs

Concepts clés

- **class** : Définit un modèle d'objet
- **constructor()** : Méthode spéciale appelée avec **new**
- **this** : Référence à l'instance courante
- **new** : Crée une nouvelle instance de la classe

```
1 export default class Author {
2   constructor(id, name, yearBorn) {
3     this.id = id;
4     this.name = name;
5     this.yearBorn = yearBorn;
6   }
7
8   toString() {
9     return '->>>> Author : ' + this.id + ' - ' +
10      this.name + ' - ' + this.yearBorn;
11   }
12 }
```

Listing 5 – Classe simple - Author

Utilisation

```
1 let a1 = new Author(101, "Brendan Eich", 1961);
2 console.log('a1 = ' + a1); // Appelle toString() automatiquement
```

2.2 Propriétés Privées (Encapsulation)

Propriétés privées

- Préfixe **#** pour déclarer une propriété privée
- Inaccessible directement depuis l'extérieur
- Nécessite des getters/setters pour y accéder

```
1 export default class Point {
2   #id = 100; // Propriété privée
3
4   constructor(x = 0, y = 0, id = 100) {
5     this.x = x;
6     this.y = y;
```

```

7      this.#id = id;
8  }
9
10 // Methode publique pour acceder la propriete priv e
11 getId() {
12     return this.#id;
13 }
14 }

```

Listing 6 – Propriété privée avec #

2.3 Getters et Setters

Accesseurs

- **get** : Permet de lire une propriété (comme un attribut)
- **set** : Permet de modifier une propriété (comme un attribut)
- Syntaxe naturelle : `obj.prop` au lieu de `obj.getProp()`

```

1 export default class Point {
2     #id = 100;
3
4     // GETTER : Lecture
5     get id() {
6         console.log('>> get id()');
7         return this.#id;
8     }
9
10    // SETTER : criture
11    set id(value) {
12        console.log('>> set id()');
13        this.#id = value;
14    }
15 }

```

Listing 7 – Getters et Setters

Utilisation des getters/setters

```

1 let p1 = new Point(20, 30, 100);
2
3 // Lecture (appelle automatiquement le getter)
4 console.log('p1.id : ' + p1.id);
5
6 // criture (appelle automatiquement le setter)
7 p1.id = 220;
8 console.log('p1.id devient : ' + p1.id);

```

2.4 Membres Statiques

Statique vs Instance

- **static** : Appartient à la classe, pas aux instances
- Accessible via `ClassName.property` ou `ClassName.method()`
- Ne peut pas accéder aux propriétés d'instance (`this.x = undefined`)

```
1 export default class Point {
2   static MAX_X = 1000;
3   static MAX_Y = 1000;
4
5   constructor(x = 0, y = 0) {
6     this.x = x;
7     this.y = y;
8   }
9
10  static info() {
11    console.log('Classe Point, MAX_X = ' + Point.MAX_X);
12    console.log('this.x : ' + this.x); // undefined !
13  }
14 }
```

Listing 8 – Propriétés et méthodes statiques

Utilisation du statique

```
1 console.log('Point.MAX_X = ' + Point.MAX_X); // OK
2 Point.info(); // OK
3
4 let p1 = new Point(20, 30);
5 console.log(p1.MAX_X); // FAUX ! undefined
```

2.5 Héritage

Héritage

- **extends** : Une classe hérite d'une autre
- **super()** : Appel du constructeur de la classe mère
- La classe fille hérite de toutes les propriétés et méthodes publiques

```
1 import Point from "./Point.js";
2
3 export default class City extends Point {
4   static counter = 100;
5
6   constructor(name, x = 0, y = 0) {
7     super(x, y, City.counter); // Appel au constructeur parent
8     this.name = name;
9     City.counter++;
10  }
11
12  toString() {
```

```

13     return this.name + " - " + this.id +
14           ' (' + this.x + ', ' + this.y + ')';
15   }
16 }

```

Listing 9 – Héritage - City extends Point

Utilisation de l'héritage

```

1 let c1 = new City('F s', 3000, 54300);
2 let c2 = new City('Rabat', 41000, 52100);
3
4 c1.print(); // Methode héritée de Point
5 console.log('c1: ' + c1); // toString() red fini

```

3 Architecture en Couches

3.1 Structure du Projet

Organisation

```

projet/
  config/
    mysql.config.js      # Configuration BDD
  models/
    Author.js           # Classe Author
    Point.js            # Classe Point
    City.js             # Classe City
  repositories/
    author.repository.js # Accès JSON
    author.mysql.repository.js # Accès MySQL
  services/
    biblio.service.js    # Logique métier
  data/
    authors.json         # Données JSON
    examples.js          # Tests
    examples.mysql.js    # Tests MySQL
    main.js              # Point d'entrée

```

3.2 Couche Repository (Accès aux données)

3.2.1 Repository MySQL

```

1 import db from "../config/mysql.config.js";
2
3 const tablename = 'authors';
4
5 export async function selectAll() {
6   let query = `SELECT * FROM ${tablename}`;
7   let result = await db.query(query);
8 }

```

```

9      // result[0] = rows, result[1] = fields
10     let authors = result[0].map(row => ({
11         id: row.Au_ID,
12         name: row.Author,
13         yearBorn: row.Year_Born
14     }));
15
16     return authors;
17 }

```

Listing 10 – Repository MySQL - author.mysql.repository.js

3.2.2 Repository JSON

```

1 import fs from 'fs';
2 const filename = 'data/authors.json';
3
4 export async function selectAll() {
5     // Lecture asynchrone du fichier
6     let data = await fs.promises.readFile(filename);
7
8     // Conversion JSON -> Objet JavaScript
9     let authors = await JSON.parse(data);
10
11     return authors;
12 }

```

Listing 11 – Repository JSON - author.repository.js

3.3 Couche Service (Logique métier)

```

1 import { selectAll } from "../repositories/author.repository.js";
2 import Author from "../models/Author.js";
3
4 export async function getAllAuthors() {
5     // 1. Récupérer les données brutes
6     let authors = await selectAll();
7
8     // 2. Transformer en objets Author
9     let result = authors.map((obj) =>
10         new Author(obj.id, obj.name, obj.yearBorn)
11     );
12
13     return result;
14 }

```

Listing 12 – Service - biblio.service.js

3.4 Couche Configuration (BDD)

```

1 import mysql from 'mysql2/promise';
2
3 const params = {
4     "host": "localhost",
5     "user": "root",
6     "password": "",
7     "database": "Biblio"

```



```

8 };
9
10 // Cr ation d'un pool de connexions
11 const db = mysql.createPool(params);
12
13 export default db;

```

Listing 13 – Configuration MySQL - mysql.config.js

4 Programmation Asynchrone

4.1 Callbacks, Promises et Async/Await

Concepts asynchrones

- **Callback** : Fonction passée en paramètre (ancien modèle)
- **Promise** : Objet qui promet une valeur future (resolve/reject)
- **async/await** : Syntaxe moderne pour gérer les Promises

4.2 Les Promises

```

1 import {selectAll} from './repositories/author.repository.js';
2
3 selectAll()
4   .then(authors => {
5     // Succ s
6     console.log('Nb. d'auteurs : ${authors.length}');
7     for (let a of authors) {
8       console.log(' - ', a);
9     }
10  })
11  .catch(err => {
12    // Erreur
13    console.log('>> err :', err);
14  });

```

Listing 14 – Utilisation de .then() et .catch()

4.3 Async/Await

Async/Await

- **async** : Déclare une fonction asynchrone (retourne une Promise)
- **await** : Attend la résolution d'une Promise
- Rend le code asynchrone plus lisible (synchrone en apparence)

```

1 import fs from 'fs';
2
3 export async function selectAll() {
4   // await bloque jusqu' la fin de la lecture
5   let data = await fs.promises.readFile(filename);
6

```

```
7 // await bloque jusqu' la conversion
8 let authors = await JSON.parse(data);
9
10 return authors;
11 }
```

Listing 15 – Async/Await - Lecture fichier JSON

Règles importantes

- `await` ne peut être utilisé que dans une fonction `async`
- Toute fonction déclarée `async` retourne automatiquement une Promise
- `await` ne bloque que la fonction asynchrone, pas tout le programme

5 Gestion des Erreurs et Fermeture

5.1 Gestion des erreurs avec `.catch()`

```
1 import db from './config/mysql.config.js';
2 import {selectAll} from './repositories/author.mysql.repository.js';
3
4 export function mysqlTest01() {
5     selectAll()
6         .then((result) => {
7             console.log('>> result :', result);
8             db.end(); // Fermer la connexion
9         })
10        .catch((err) => {
11            console.log('>> err :', err);
12        });
13 }
```

Listing 16 – Gestion d'erreur MySQL

Fermeture de connexion

- Toujours appeler `db.end()` après usage pour libérer les ressources
- Sinon, le programme reste en attente de connexions ouvertes

6 Fonction Constructeur (Avant ES6)

Historique

Avant ES6, on utilisait des fonctions constructeurs et le prototype pour créer des classes.

```
1 // Fonction constructeur
2 export default function Point(x = 0, y = 0) {
3     this.x = x;
4     this.y = y;
5 }
6
7 // Ajout de méthodes via le prototype
```

```

8 Point.prototype.print = function() {
9   console.log('Point : ' + this.x + ', ' + this.y);
10 };
11
12 // Tous les objets Point h ritent de Point.prototype
13 // Point.prototype h rite de Object

```

Listing 17 – Fonction constructeur (ancien modèle)

À retenir

- Cette approche est obsolète depuis ES6
- Utiliser `class` à la place (plus clair et plus simple)
- Comprendre le concept pour savoir lire d'ancien code

7 Récapitulatif des Concepts

Concept	Syntaxe/Utilisation
Export par défaut	<code>export default class X {}</code>
Export nommé	<code>export function f() {}</code>
Import par défaut	<code>import X from './X.js'</code>
Import nommé	<code>import {f} from './X.js'</code>
Classe	<code>class X { constructor() {} }</code>
Propriété privée	<code>#prop = value</code>
Getter	<code>get prop() { return this.#prop; }</code>
Setter	<code>set prop(v) { this.#prop = v; }</code>
Statique	<code>static prop = value</code>
Héritage	<code>class Y extends X { super() }</code>
Async	<code>async function f() { await ... }</code>
Promise	<code>f().then().catch()</code>

TABLE 1 – Récapitulatif des concepts

8 Exemples d'Utilisation

8.1 Création d'objets

```

1 import Point from './models/Point.js';
2 import City from './models/City.js';
3 import Author from './models/Author.js';
4
5 // Cr ation d'un Point
6 let p1 = new Point(20, 33);
7 p1.print(); // Point(20, 33)
8
9 // Acc s aux propri t s priv es via getter
10 console.log('p1.id : ' + p1.id);
11
12 // Modification via setter
13 p1.id = 220;

```

```
14
15 // Acc s aux propri t s statiques
16 console.log('Point.MAX_X = ' + Point.MAX_X);
17
18 // Cr ation d'une City (h rite de Point)
19 let c1 = new City('F s', 3000, 54300);
20 console.log('c1: ' + c1);
21
22 // Cr ation d'un Author
23 let a1 = new Author(101, "Brendan Eich", 1961);
24 console.log('a1 = ' + a1);
```

Listing 18 – Exemple complet

8.2 Appel aux services

```
1 import { getAllAuthors } from "../services/biblio.service.js";
2
3 getAllAuthors()
4   .then(authors => {
5     console.log('Nb. d'auteurs : ${authors.length}');
6     for (let a of authors) {
7       console.log(' - ' + a);
8     }
9   });
```

Listing 19 – Appel au service

9 Points Importants à Retenir

À retenir absolument

1. **Modules** : export default (1 seul) vs export (plusieurs)
2. **Propriétés privées** : Préfixe #, accessibles uniquement via getters/setters
3. **Statique** : Appartient à la classe, pas aux instances (Point.MAX_X)
4. **Héritage** : extends + super() dans le constructeur
5. **Async/Await** : async pour déclarer, await pour attendre
6. **Promises** : .then() pour le succès, .catch() pour les erreurs
7. **Architecture** : Config → Repository → Service → Exemples → Main
8. **Fermeture** : Toujours db.end() après usage de MySQL

Conclusion

Cette fiche couvre l'ensemble des concepts du cours. Pour réussir l'examen :

- Maîtriser la syntaxe des classes ES6 (propriétés privées, getters/setters, statique)
- Comprendre l'héritage et l'appel à super()
- Savoir utiliser async/await et les Promises
- Connaître l'architecture en couches (Repository, Service, etc.)

— Pratiquer avec des exemples concrets

Bon courage pour tes révisions !